01 / Architecture et périmètre

Référence technique : dev au commit 4af49f6, 1er novembre 2026. Vadim Gavet, Jonathan Cahoreau et Brice Fontaine.

NoWave permet de publier des observations maritimes, de les consulter sur une carte et de recevoir des alertes à proximité. Trois catégories : animaux marins, obstacles et pollution.

Composant	Rôle
Flutter / Dart	App Android et iOS : compte, GPS, caméra, carte, publication et alertes.
FastAPI / Python	Contrats HTTP, règles métier, contrôle des accès et adaptation des fournisseurs.
PostgreSQL / PostGIS	Profils, appareils, signalements, positions et file de notifications.
Firebase Auth / FCM	Identité e-mail/mot de passe et transport des messages push.
Mapbox	Fond Standard, globe et rendu des couches marines et des signalements.
Scaleway Object Storage	Bucket privé des photos ; lecture par URL signée.
Web / Jinja / Nginx	Site public, carte et pages d’administration.
Docker / VPS / Cloudflare	Services déployés, réseau privé SQL et exposition HTTPS.

Circulation des données

Le mobile envoie un token Firebase à l’API. L’API applique les règles NoWave et écrit dans PostGIS. Les signalements deviennent des tuiles MVT authentifiées ; le worker envoie les alertes retenues par PostGIS. Les données marines publiques passent par leurs adaptateurs et caches, séparément des signalements.

Organisation du dépôt

backend/ : API, domaine, adaptateurs, migrations et workers. mobile/ : app Flutter. web/ : site et assets partagés. deploy/ : images et exploitation. docs/ : guides et archives. .github/workflows/ : tests, audits et livraison.

Périmètre livré

Compte et profil, bateau facultatif, publication manuelle ou photo, carte mondiale, alertes reçues et modération. L’édition, le retrait par l’auteur et l’historique personnel des signalements ne sont pas disponibles.

Téléchargement : no-wave.fr/downloads/mobile/. Les observations communautaires et les positions estimées complètent les informations nautiques officielles.

02 / Comptes et appareils

Créer un profil

Firebase crée l’identité et vérifie l’e-mail. POST /api/v1/users/me crée ensuite le profil NoWave avec un pseudo unique, limité à 100 caractères. GET /users/me est une lecture et renvoie 404 avant cette initialisation. Le backend extrait lui-même l’UID et l’e-mail du token vérifié.

Un nouveau profil reçoit le rôle user. Le compte désigné pour l’administration sera configuré par le responsable du projet ; l’inscription publique n’accorde pas automatiquement les droits admin.

Accès et session

Les routes métier exigent Authorization: Bearer avec un Firebase ID token, puis un profil actif. Les routes admin ajoutent le contrôle du rôle admin. Une panne d’authentification fournisseur donne 503 authentication_unavailable ; un token refusé donne 401.

Un compte suspended reçoit 403 user_inactive sur les routes métier. Les lectures et opérations nécessaires au diagnostic, à la déconnexion et à la suppression du compte restent accessibles selon leur contrat. Aucun mot de passe n’est conservé dans PostgreSQL.

Préférences et confidentialité

show_user_name, show_boat_info et notifications_enabled valent false par défaut. Le partage du pseudo, celui du bateau et les alertes sont indépendants. Date de naissance, nationalité et e-mail restent privés. La suppression du compte retire ses données personnelles et ses appareils ; ses signalements restent anonymisés avec author_id = NULL.

Enregistrer le téléphone

PUT /devices/current reçoit installation_id, platform et fcm_token. L’installation appartient à un seul compte. Un token absent est transmis à null. La position et la désactivation utilisent X-Installation-ID.

PUT /devices/current/position remplace uniquement la dernière position. Précision finie entre 0 et 50 m ; cap entre 0 inclus et 360 exclus ; measured_at ne dépasse pas l’heure serveur de plus de cinq minutes. Une mesure plus ancienne est refusée.

Changer de compte

Le mobile arrête ses envois, attend le nettoyage serveur et désactive l’appareil avant de fermer la session Firebase. Le token, la position et les tâches en attente sont nettoyés dans une transaction. Un nouvel identifiant d’installation est créé pour le compte suivant. Si le nettoyage échoue, la session et l’identifiant sont conservés pour réessayer.

03 / Signalements et photos

Contrat de publication

POST /api/v1/reports accepte category, positioning_mode, description facultative (250 caractères), final_position, observed_at et client_report_id. Les coordonnées sont un Point GeoJSON [longitude, latitude] en WGS84 ; les horodatages utilisent UTC.

Règle	Valeur
Catégorie	marine_animal, obstruction ou pollution
Mode	manual ou photo
Observation	Pas dans le futur ; strictement moins de 24 h avant l'heure serveur
Expiration	observed_at + 24 h ; seules les observations actives non expirées sont visibles
Idempotence	UNIQUE(author_id, client_report_id) ; garder le même UUID lors d'un réessai
Version	1 à la création ; la modération ne l'incrémente pas

Parcours photo

La capture fige le GPS, sa précision, l'azimut, l'inclinaison et les métadonnées caméra. POST /position-estimates propose une position sans enregistrer de signalement. L'utilisateur confirme ou corrige toujours le point sur la carte. Si le calcul échoue, il peut le placer lui-même en gardant le mode photo.

L'azimut suit l'axe optique de la caméra arrière par rapport au nord vrai. Inclinaison : zéro à l'horizon, positive vers le haut. La hauteur par défaut est 2,5 m avec une incertitude de 0,5 m ; cette estimation non calibrée demande une validation sur appareil.

Upload et lecture

Le mobile crée d'abord le signalement, puis envoie le champ multipart file à POST /reports/{id}/photo. JPEG uniquement : 1 à 500 000 octets, 1600 px par côté, 2 560 000 pixels maximum et aucun EXIF. La requête multipart complète est limitée à 512 000 octets.

Premier upload : 201. Même JPEG répété : 200. JPEG différent après finalisation : 409. Un échec de stockage laisse le signalement publié et la photo failed ; réessayer sur le même report_id. Le formulaire mobile ne constitue pas un brouillon persistant.

La lecture utilise une URL signée de cinq minutes sans token Firebase envoyé au stockage. Masquer une photo bloque les nouvelles URL, y compris pour l'admin ; celles déjà émises expirent normalement.

Nettoyage

photo_cleanup passe toutes les cinq minutes et supprime uniquement les objets non référencés âgés d'au moins une heure. Upload et nettoyage partagent le verrou SQL de la photo. Les objets référencés sont conservés, même si la photo est masquée ou le signalement expiré. Le bucket est privé et sans versioning.

04 / Alertes et position GPS

Sélection transactionnelle

La création du signalement écrit aussi alert_history et notifications dans la même transaction. PostGIS sélectionne les autres utilisateurs actifs ayant activé les alertes, avec un appareil actif, un token FCM et une position de précision 0 à 50 m à 18 520 m au plus (10 NM). Ni le cap ni l’âge de la dernière position ne filtrent la sélection.

L’historique garde une entrée par utilisateur/signalement/version et la plus petite distance de ses appareils éligibles. La file garde une tâche par appareil/signalement/version. Ces contraintes évitent de doubler les alertes lors d’un rejeu de publication.

Worker FCM

Le worker relit visibilité, expiration, version et éligibilité avant l’envoi. Les erreurs temporaires sont reprises après 1, 5, 15 et 30 minutes, puis 60 minutes ; un Retry-After plus long est respecté. Les tentatives s’arrêtent à l’expiration. Seule l’erreur UNREGISTERED efface le token de l’appareil.

sent signifie que FCM a accepté le message, sans preuve de livraison ou de lecture. Un crash avant commit peut produire un nouvel envoi avec les mêmes identifiants. Les tâches terminales sont purgées après sept jours ; l’historique utilisateur reste disponible jusqu’à la suppression du compte.

Payload et navigation

Les valeurs data sont des chaînes. Champs obligatoires : alert_id, report_id, report_version, category, position_dms, distance_m, expires_at. Champs facultatifs : position (Point GeoJSON sérialisé) et alerted_at. Si position est présent, il doit être cohérent avec position_dms.

Le push identifie l’alerte ; le mobile relit GET /reports/{id} avant de cadrer la carte. Un appui ouvre la carte principale et son repère, puis le détail sur demande. Le formulaire de publication en cours est conservé. Les doublons et les messages d’une ancienne session sont ignorés.

Suivi mobile

Première mesure exploitable envoyée immédiatement, puis au plus une fois par minute. Un déplacement de moins de 50 m n’est pas renvoyé, sauf précision au moins deux fois meilleure. Ce filtre ne modifie pas le flux local de la carte et de la caméra.

Les alertes activées permettent le suivi en arrière-plan ; une fermeture forcée ou un arrêt système l’interrompt jusqu’à la réouverture. Déplacer la carte ne change jamais la position GPS enregistrée. L’historique GET /users/me/alerts est paginé et ne porte pas d’état lu/non lu côté serveur.

05 / Modèle de données

PostgreSQL/PostGIS utilise des UUID, timestamptz et geography(Point,4326). Les migrations Alembic définissent le schéma réel. Les couches MVT et leurs clusters sont calculés ; ils ne forment pas des tables métier.

Table	Données et relations essentielles
users	UID Firebase, e-mail et pseudo uniques ; rôle, statut et trois préférences privées par défaut.
boats	Bateau facultatif ; user_id unique, donc un bateau maximum par compte.
devices	Installation unique, utilisateur, plateforme, token FCM et activité.
device_positions	Une position courante par appareil ; précision, cap et horodatages.
reports	Auteur facultatif après suppression du compte ; catégorie, position finale, observation, expiration, état et version.
report_positioning	Au plus une ligne par signalement photo ; mesures de capture et estimation facultative.
report_photos	Une ligne par signalement photo ; upload_status, object_key, taille, type et hidden_at.
notifications	File persistante par appareil et version ; état, tentatives et prochaine échéance.
alert_history	Instantané d’alerte par utilisateur et version ; catégorie, position et distance minimale.
moderation_actions	Action immuable, admin, cible, motif et changement d’état.
map_cache_revision	Ligne technique unique ; UUID renouvelé après modification des signalements.

Contraintes à conserver

UNIQUE(author_id, client_report_id), UNIQUE(report_id, device_id, report_version) et UNIQUE(user_id, report_id, report_version) garantissent l’idempotence de la publication, de la file et de l’historique.

Le mode photo crée reports, report_positioning et report_photos dans une même transaction ; le mode manuel ne crée aucune ligne photo. Des contraintes différées vérifient cette cohérence au commit. Les positions de signalement et d’appareil disposent d’index GiST.

La suppression d’un parent supprime ses dépendances selon les clés étrangères. Les liens auteur des signalements et les liens d’audit utilisent SET NULL pour garder l’historique. Une action de modération et son audit sont validés ensemble.

06 / Cartographie marine

Trois sources distinctes

Mapbox Standard fournit le fond globe. Les signalements viennent des MVT NoWave authentifiées produites depuis PostGIS. Les données marines publiques sont normalisées et fusionnées par FastAPI à partir de Shom, NOAA, IGN, OSM, EMODnet et GEBCO, avec leurs crédits.

Le fond mondial GEBCO 2026 Z0-Z6 est un bundle WebP statique servi par Nginx ; il ne génère ni sondes ni courbes de profondeur. Le détail régional EMODnet se superpose dès Z5 dans sa couverture.

Couche	Chargement et affichage
Bathymétrie	Raster dès Z5
Ports	Préparation Z6, affichage Z8, enrichissement Z10
Balisage	Préparation Z10 ; fondu de 0 % à Z10 à 100 % à Z12
Profondeurs	Préparation Z12, affichage Z13 ; courbes conservées sur la grille Z12

Conserver les objets utiles

MarineViewport suit l’emprise visible, y compris à l’antiméridien, et donne priorité au centre. Une réponse complète remplace sa zone ; une réponse complète vide confirme une absence. Une réponse partielle ou une erreur ne retire pas une géométrie connue avant sa relève complète.

Bornes du viewport : 64 cellules, deux téléchargements GeoJSON simultanés, 64 snapshots / 12 Mio encodés et 8 000 points conservés / 4 Mio. Les buffers natifs et GPU sont hors de ces budgets. Les demandes obsolètes sont annulées et les marges sont sacrifiées avant le centre.

Cache et panne

Flux	Politique HTTP
Signalements MVT	private, max-age=15, must-revalidate ; Vary: Authorization
Vecteurs marins hors sondes	300 s normalement, 15 s si dégradés
Sondes	no-store
Bathymétrie PNG	24 h si complète ; contexte incomplet : no-store
Fournisseur indisponible	503 JSON, no-store, Retry-After: 15, statut unavailable
Vide valide / hors couverture	200 ; ne doit pas être confondu avec une panne

MARINE_CACHE_DIR doit être inscriptible dans le conteneur API. Le volume marine-cache conserve les caches SQLite. Les crédits et licences restent dans les guides docs/architecture/ et dans /api/v1/map/marine-sources. Une couverture partielle et un cache ne garantissent pas la disponibilité du fournisseur.

07 / API métier et administration

Préfixe commun : /api/v1. Les routes ci-dessous utilisent le token Firebase ; les routes /admin exigent en plus un profil actif de rôle admin. Les pages HTML /admin sont distinctes.

Méthode et chemin	Usage
GET /users/me	Lire son profil sans création implicite
POST /users/me	Créer le profil après vérification de l’e-mail
PATCH /users/me	Mettre à jour les champs autorisés
DELETE /users/me	Supprimer le compte et anonymiser les signalements
GET /users/me/boat	Lire son bateau
POST /users/me/boat	Créer son bateau
PATCH /users/me/boat	Modifier les seuls champs fournis
DELETE /users/me/boat	Supprimer son bateau
PUT /devices/current	Enregistrer ou actualiser son appareil
DELETE /devices/current	Désactiver l’appareil ; X-Installation-ID requis
PUT /devices/current/position	Remplacer la position ; X-Installation-ID requis
POST /reports	Créer un signalement
GET /reports/{id}	Lire un signalement actif et non expiré
POST /reports/{id}/photo	Envoyer le JPEG dans le champ multipart file
POST /position-estimates	Calculer une estimation sans la persister
GET /users/me/alerts	Historique d’alertes reçues paginé
GET /admin/reports	Lister les signalements à modérer
GET /admin/reports/{id}	Lire le détail, y compris expiré ou retiré
PATCH /admin/reports/{id}	Restaurer avec status=active et reason
DELETE /admin/reports/{id}	Retirer avec un motif JSON ; réponse 204
PATCH /admin/reports/{id}/photo	Masquer avec hidden=true et reason
GET /admin/users	Lister les comptes
PATCH /admin/users/{id}	Suspendre ou réactiver avec reason
GET /admin/moderation-actions	Lire l’audit ; aucune écriture exposée

Pagination admin : page=1, limit=20 par défaut, maximum 100. Une page au-delà de la fin reste valide avec items vide. Un motif absent ou vide donne 422 ; un conflit d’état donne 409 moderation_conflict.

08 / API carte et erreurs

Méthode et chemin	Accès / format
GET /health	Public ; santé du processus
GET /health/ready	Public ; connexion PostgreSQL
GET /version	Public ; révision de déploiement
GET /api/v1/map/tiles/{z}/{x}/{y}.mvt	Firebase ; MVT des signalements
GET /api/v1/map/web-config	Public ; jeton Mapbox pk.* ou chaîne vide, no-store
GET /api/v1/map/observations	Public ; observations actives, no-store
GET /api/v1/map/seamarks/{z}/{x}/{y}.pbf	Public ; balisage MVT
GET /api/v1/map/port-infrastructure/{z}/{x}/{y}.pbf	Public ; équipements portuaires MVT
GET /api/v1/map/harbours/{z}/{x}/{y}.pbf	Public ; ports MVT
GET /api/v1/map/depth-contours/{z}/{x}/{y}.pbf	Public ; isobathes MVT
GET /api/v1/map/soundings/{z}/{x}/{y}.pbf	Public ; sondes MVT, no-store
GET /api/v1/map/underwater-hazards/{z}/{x}/{y}.pbf	Public ; dangers MVT
GET /api/v1/map/marine-features/{family}/{z}/{x}/{y}.geojson	Public ; snapshots de famille
GET /api/v1/map/marine-sources	Public ; crédits et provenance
GET /api/v1/map/bathymetry/{z}/{x}/{y}.png	Public ; bathymétrie PNG

Fil public

GET /map/observations renvoie items, has_more et next_cursor. Filtres : west/south/east/north ensemble, category, q (120 caractères), limit (1-500, défaut 200) et cursor. west > east traverse l’antiméridien. Garder les mêmes filtres à la page suivante ; tri observed_at DESC, id DESC.

Le fil public exclut les observations expirées ou retirées. Il publie identifiant, catégorie, coordonnées, dates, description non vide et pseudo si autorisé ; aucun e-mail, identifiant d’auteur, bateau ou photo.

Erreurs

Format JSON : error.code, error.message et error.details. 401 : identité absente ou refusée. 403 : droits insuffisants ou compte inactif. 404 : ressource absente. 409 : conflit d’unicité ou d’état. 422 : paramètres invalides. 429 : débit dépassé. 503 : dépendance indisponible.

Les bornes de zoom varient selon la famille ; une combinaison refusée donne 422. Pour les valeurs exactes, consulter backend/app/api/routes/map_tiles.py. L’API ne fournit ni PATCH/DELETE auteur sur /reports/{id}, ni GET /users/me/reports.

09 / Web et modération

Carte partagée

Le site public et l’administration utilisent le même composant map-view.js. L’entrée admin-map.js ajoute les liens et actions protégés. Le build produit les assets publics, les assets admin et /assets/nowave.css. Le frontal doit servir les assets du même build que les pages admin.

La carte publique et la carte d’accueil admin chargent le fil public par pages de 50. Les détails et actions privés passent par les contrôles administrateur. Une erreur API reste un état indisponible, sans substitution par de fausses observations.

Session web

FastAPI/Jinja sert /admin. La connexion exige une identité Firebase vérifiée, un profil actif de rôle admin et FIREBASE_WEB_API_KEY ou sa variante _FILE. La session dure huit heures dans un cookie Secure, HttpOnly et SameSite=Strict. Chaque requête revérifie la révocation Firebase et relit le rôle NoWave.

Les formulaires POST contrôlent un token CSRF ; Jinja échappe les contenus. Les pages privées sont no-store. La déconnexion efface les cookies du navigateur sans révoquer la session mobile. Sans clé Firebase Web API, la connexion web renvoie 503.

Actions et audit

Action	Effet
Retirer un signalement	Suppression logique ; le détail admin reste consultable.
Restaurer un signalement	Conserve observed_at et expires_at ; pas de nouvelle alerte.
Masquer une photo	Conserve l’upload et le signalement ; bloque les nouvelles URL signées.
Suspendre un compte	Bloque les futures publications et alertes sans retirer ses signalements.
Réactiver un compte	Rend les fonctions accessibles si les autres contrôles sont satisfaits.

Chaque action exige un motif non vide et écrit une entrée d’audit dans la même transaction. L’audit reste en lecture seule ; ses liens peuvent devenir null après suppression d’un compte ou d’une cible.

Navigation et données privées

Les listes HTML acceptent recherche, statut et pagination. Les signalements d’un utilisateur sont paginés par 20. Les retours gardent cadrage et filtres dans l’onglet. L’admin peut lire les données nécessaires à la modération indépendamment du partage public ; une photo déjà masquée ne reçoit cependant aucune nouvelle URL.

10 / Développement et livraison

Démarrer

Les versions des outils et les commandes détaillées sont dans docs/TESTING.md. Compléter .env depuis .env.example : base, Firebase, Mapbox et stockage photo. Les secrets restent hors de Git.

```
cp .env.example .env
docker compose config --quiet
docker compose up --build -d --wait
```

Le site et /admin sont sur https://localhost:8443 avec un certificat local. Le backend est accessible en boucle locale sur :8000. Les migrations s'appliquent au démarrage. La base existante est conservée.

Pour Flutter, créer mobile/.env.json depuis l'exemple, renseigner MAPBOX_ACCESS_TOKEN et API_BASE_URL, puis lancer flutter run avec --dart-define-from-file=.env.json. MARINE_API_BASE_URL peut isoler les seules couches publiques ; vide, elle reprend l'API principale. Changer ces valeurs exige une nouvelle compilation.

Contrôles

checks.yml lance Python, Flutter, PostGIS, Web et Admin Web. La suite PostGIS inclut les parcours admin Playwright avec une base isolée et les assets compilés. Les audits des dépendances et des images suivent via production.yml.

```
ruff check backend deploy/hostinger mobile/tool
python -m pytest backend/tests/api backend/tests/unit -q
(cd mobile && flutter analyze --no-pub && flutter test --no-pub)
npm test --prefix web
npm run build --prefix web
```

Ces commandes supposent les dépendances installées. Pour PostGIS, les audits et les tests navigateur, reprendre les commandes dédiées de docs/TESTING.md ; une base partagée ou de production ne sert jamais de base de test.

Branches et déploiement

Branche de travail vers dev par PR et revue ; dev vers main pour la version stable. Après réussite des contrôles et audits, un push sur dev peut déployer la révision sur le VPS. Les PR et les lancements manuels ne déploient pas. Les règles d'approbation GitHub se vérifient dans la configuration du dépôt.

Le VPS exécute API, web, PostgreSQL/PostGIS et workers. Le tunnel Cloudflare publie les services ; PostgreSQL reste privé. /version et les logs du workflow permettent de vérifier la révision disponible. Les procédures de sauvegarde et de retour arrière sont dans deploy/README.md.

Limites de validation

Une CI verte ne prouve pas la réception FCM, la justesse des capteurs, la disponibilité des fournisseurs ou le bon fonctionnement du VPS. Les essais Android/iOS et le contrôle public de la version déployée sont séparés. Les rapports du 29 octobre restent des archives datées, sans nouvelle exécution revendiquée ici.